

Department of Computer Science and Engineering

Assignment Questions

Course Code & Name: CSA0621 –Design and Analysis of Algorithms

Particular	Details
Department:	Computer Science and Engineering
Programme:	B E.CSE(AI)
Course Code & Course Name	CSA0621 - Design and Analysis of Algorithms
Academic Year / Batch	2 nd Year
Faculty Name	Dr G Nagarajan
Assignment Title	Development of an Efficient Algorithmic Solution for Large-Scale Data Processing
Date of Issue	01-09-2026
Date of Submission	01-09-2026
Maximum Marks	100

Course Outcome(s) – CO	CO1: Analyse the efficiency of recursive and non-recursive algorithms using asymptotic notations (Big-O, Ω, Θ) and complexity-analysis methods such as the master theorem, substitution, and iteration. CO2: Apply brute-force techniques such as sorting and searching to solve computational problems. CO3: Apply divide-and-conquer techniques to design efficient algorithmic solutions.
Bloom's Taxonomy Level	L4 – Analyse; L5 – Evaluate; L6 – Create
SDG Mapping (SDG 1-17)	SDG 9 – Industry, Innovation and Infrastructure
Industry / Societal Relevance	Smart Infrastructure Planning and Geospatial Data Optimization

Assignment Problem / Challenge

The assignment should preferably require students to:

- Apply course concepts rather than reproduce theory.
- Identify and analyze the problem.
- Consider requirements and constraints.
- Develop or compare alternative solutions where appropriate.
- Use relevant modern engineering/computing/design tools. Interpret results.
- Make and justify engineering decisions.
- Consider appropriate technical, economic, environmental, societal, ethical, safety or sustainability factors wherever relevant.

B. Problem Statement

An infrastructure-planning agency maintains a large, publicly available geospatial dataset of facility locations — for example, the OpenFlights airports dataset (~40,000 points) or a cell-tower / weather-station dataset containing 10^4 – 10^6 coordinate records. To detect redundant or dangerously close facilities and to optimise service coverage, the agency must identify the **closest pair of locations** — the two points separated by the minimum Euclidean distance in the dataset. Develop, implement, and evaluate a solution to this Closest-Pair-of-Points problem. Implement (i) a **Brute-Force** algorithm that evaluates all $n(n-1)/2$ pairs, of order $O(n^2)$, and (ii) a **Divide-and-Conquer**

algorithm that recursively partitions the point set about the median x-coordinate and merges across the dividing strip, of order $O(n \log n)$; then develop an optimised **hybrid** that reverts to brute force below a threshold sub-problem size. Analyse the theoretical efficiency of each approach using BigO, Ω , and Θ notations, solving the recurrence $T(n) = 2T(n/2) + O(n)$ by the master theorem. Experimentally evaluate all three algorithms on the chosen dataset across increasing input sizes (for example, 10^3 to 10^6 points), measuring execution time, number of distance computations/comparisons, memory utilisation, and scalability. Based on the theoretical and experimental results, optimise the solution and justify why the proposed hybrid is most suitable for large-scale infrastructure data — reflecting Bloom’s Levels 4–6 (Analyse, Evaluate, Create) and contributing to SDG 9 (Industry, Innovation and Infrastructure).

C. Requirements and Constraints

Use a publicly available real-world coordinate dataset with enough records to test multiple input sizes (10^3 – 10^6 points). All three algorithms (brute force, divide-and-conquer, hybrid) must be run in the same computational environment for a fair comparison, on identical inputs. Report both empirical timings and operation counts, and clearly document the threshold chosen for the hybrid, along with all assumptions and limitations.

D. Student Work

1. Problem Statement and Problem Formulation

Modern infrastructure systems generate large collections of geographical coordinates representing facilities such as airports, communication towers, weather-monitoring stations and service centres. When two facilities are located unusually close to one another, the situation may indicate duplication, inefficient resource allocation or a possible planning concern. Therefore, the agency requires an efficient computational method to identify the two locations separated by the smallest distance.

The problem can be represented as a set P of n two-dimensional points, where every point corresponds to a facility location. The task is to determine a pair (P_i, P_j) such that the Euclidean distance between them is smaller than or equal to the distance between every other possible pair. The project investigates three solutions: exhaustive Brute Force, Divide-and-Conquer, and an optimised Hybrid approach. The central challenge is to preserve correctness while making the solution scalable for datasets containing thousands or potentially millions of records.

2. Objective and Expected Outcomes

The objective of this project is to design and evaluate efficient algorithms for detecting the closest pair of locations in large geospatial datasets. The work aims to establish Brute Force as a baseline, implement a Divide-and-Conquer solution with improved asymptotic efficiency, and create a Hybrid algorithm that uses different strategies according to subproblem size.

The expected outcome is a validated program that returns the same minimum distance for all correct implementations while clearly demonstrating differences in execution time, distance computations, memory consumption and scalability. The project is also expected to identify an experimentally

suitable threshold for the Hybrid algorithm and justify the final algorithmic choice using both theoretical analysis and measured results.

3. Requirements, Constraints and Assumptions

The system must accept valid coordinate records, process increasing input sizes and return the closest pair together with its minimum Euclidean distance. It must also record performance indicators such as execution time, distance evaluations and memory utilisation so that the three approaches can be compared objectively.

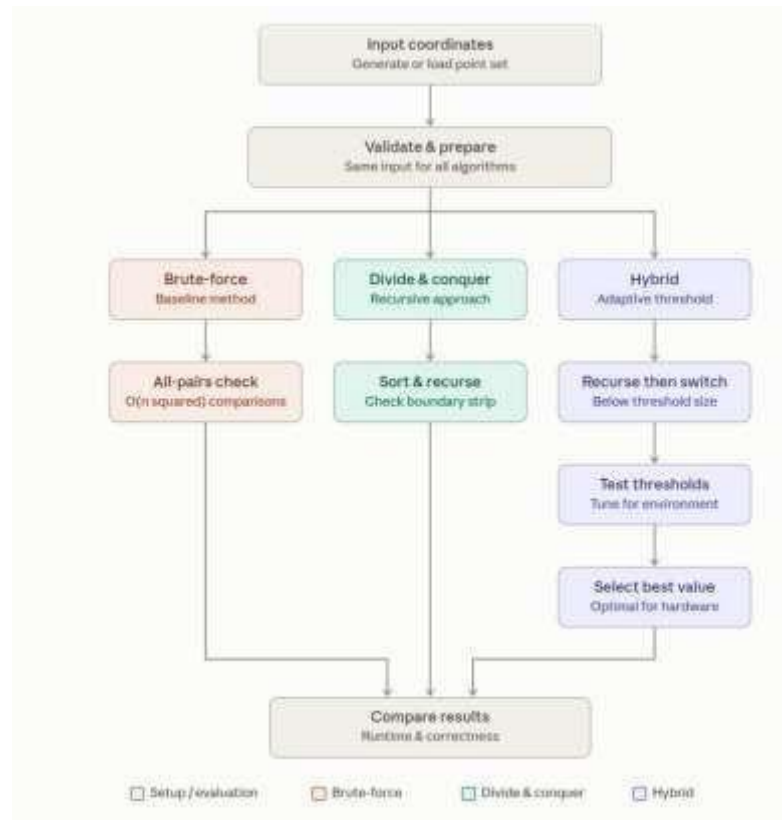
A major constraint is the quadratic growth of Brute Force, which makes full pairwise comparison impractical at very large input sizes. Experimental results are also influenced by processor speed, available memory and the characteristics of the selected dataset. The study assumes that the coordinates are represented consistently in two dimensions and that Euclidean distance is appropriate for the experimental problem formulation. Duplicate coordinates are permitted and should correctly produce a minimum distance of zero.

4. Application of Relevant Course Knowledge

The project directly applies core Design and Analysis of Algorithms concepts. Brute Force is used to create a straightforward reference solution, while Divide-and-Conquer is used to partition the point set recursively and reduce unnecessary comparisons. The merging stage uses the geometric property that only a limited number of neighbouring points in the dividing strip need to be checked after the recursive halves have been solved.

The project also applies asymptotic analysis through Big-O, Omega and Theta notation. The recurrence $T(n) = 2T(n/2) + O(n)$ is analysed using the Master Theorem, producing $\Theta(n \log n)$. Experimental analysis connects these theoretical concepts with observed program behaviour, while the Hybrid design demonstrates practical optimisation through threshold-based algorithm selection.

5. Design / Proposed Solution / Methodology



6. Pseudocode /Algorithm

Algorithm 1: BRUTE_FORCE(P)

1. Set minimum distance to infinity.
2. For each point P_i , compare it with every subsequent point P_j .
3. Calculate the Euclidean distance.
4. Replace the stored minimum when a smaller distance is found.
5. Return the closest pair and minimum distance.

Algorithm 2: DIVIDE_AND_CONQUER(P)

1. If the number of points is small, solve directly.
2. Divide the points around the median x-coordinate.
3. Recursively solve the left and right subsets.
4. Select the smaller of the two distances.
5. Construct a strip around the dividing line.
6. Check the required neighbouring points inside the strip.
7. Return the best pair found.

Algorithm 3: HYBRID_CLOSEST_PAIR(P)

1. If $\text{size}(P)$ is less than or equal to the threshold, use BRUTE_FORCE(P).
2. Otherwise apply the Divide-and-Conquer procedure.
3. Merge the recursive results using strip checking.
4. Return the minimum distance and corresponding pair.

7. Source Code and Environment / Tools Used

The implementation is developed in Python because it provides readable syntax and convenient support for experimental measurement. The program can be executed using Visual Studio Code or Python IDLE. Standard modules such as time and tracemalloc are used to measure execution time and peak memory consumption. Matplotlib can be used to create performance graphs from the collected measurements.

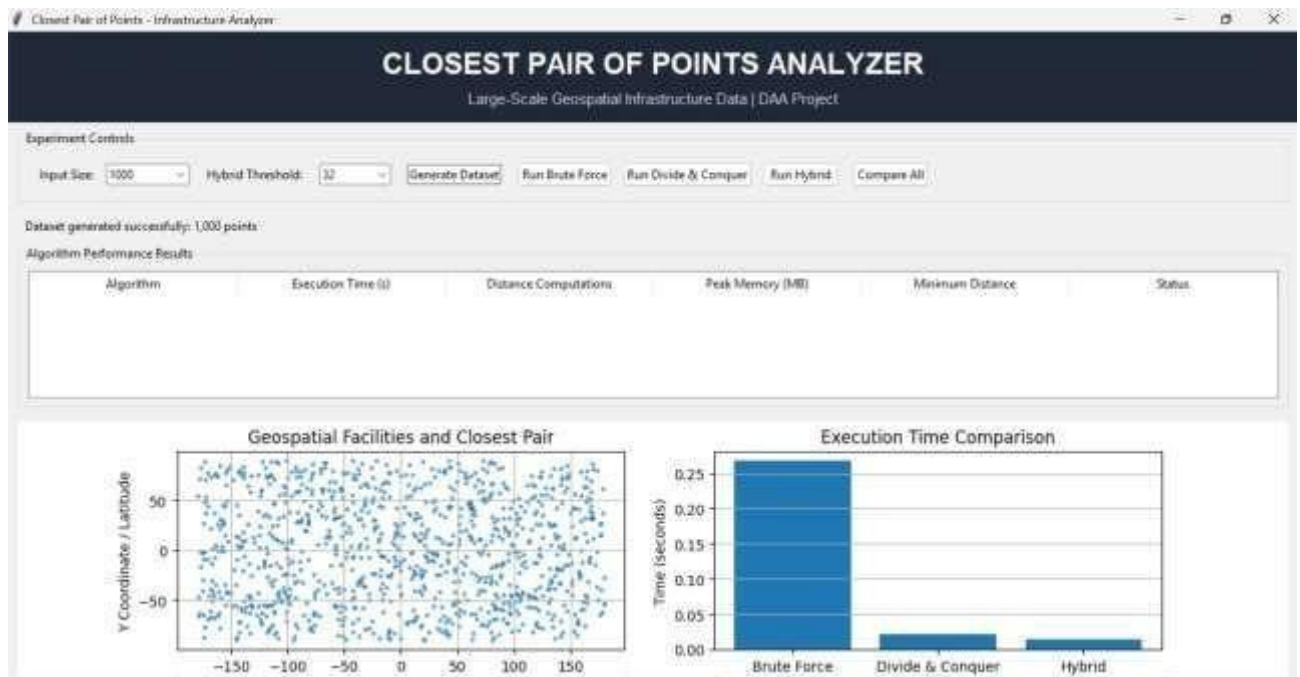
The source code is organised around separate functions for distance calculation, Brute Force, Divide-and-Conquer, Hybrid processing, dataset preparation and experiment execution. Keeping these components separate improves readability, testing and future modification.

8. Test Cases and Expected Results

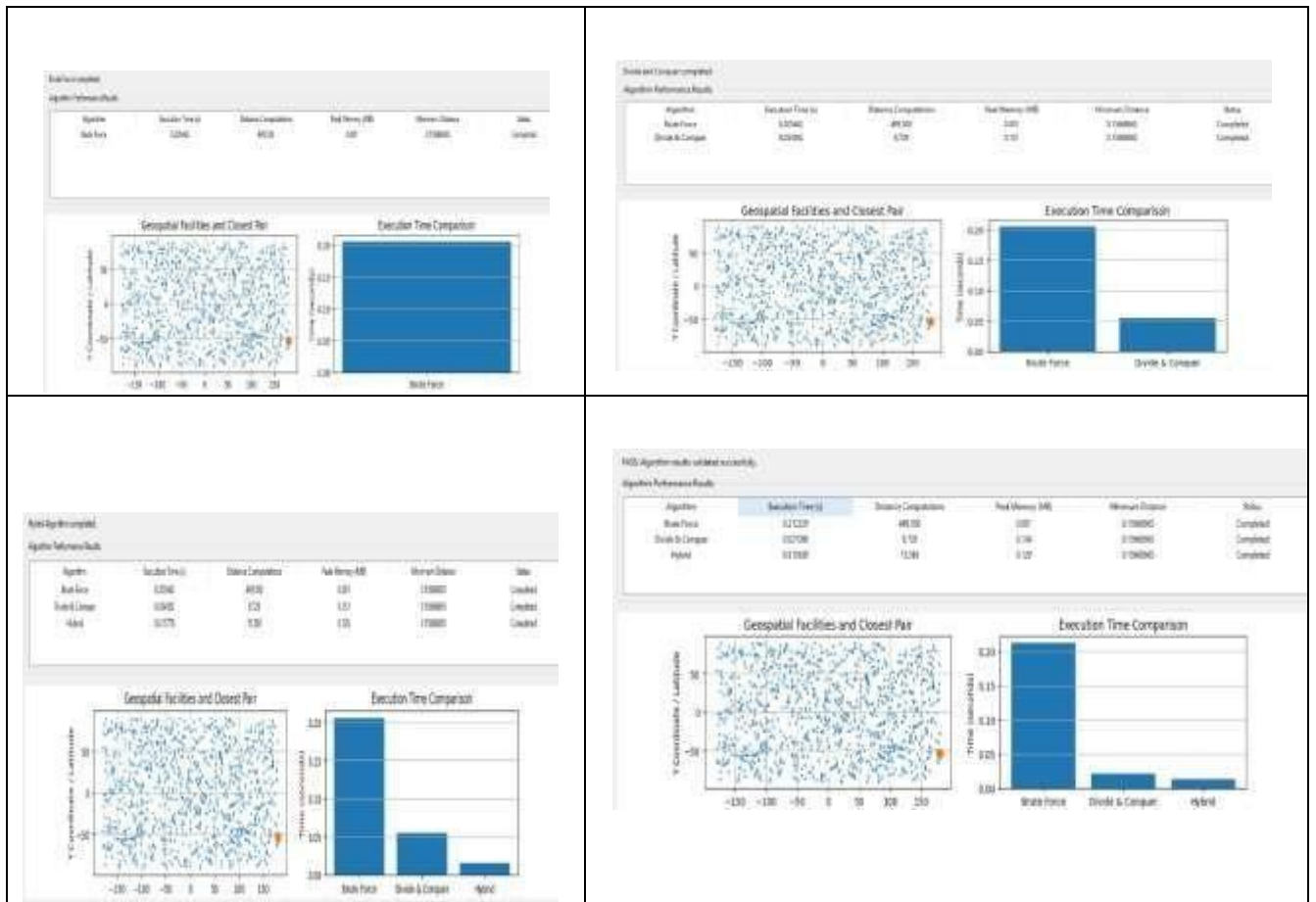
The implementation should first be tested using very small point sets where the correct answer can be verified manually. A two-point test should return the distance between those points. A dataset containing duplicate coordinates should return zero. A known sample dataset should confirm that the identified closest pair matches the manually calculated result.

For randomly generated or real datasets, validation is performed by comparing the minimum distance returned by all three algorithms. For the same input, the numerical result should be identical apart from insignificant floating-point representation differences. As input size increases, the expected performance pattern is that Brute Force grows rapidly while Divide-and-Conquer and Hybrid approaches remain substantially more scalable.

9. Execution Screenshots



10. Results and Validation – appropriate tables, graphs, performance metrics, experimental observations, or other evidence, wherever applicable



11. Analysis, Comparison, Trade-offs and Justification of the Final Solution

Brute Force has the advantage of conceptual simplicity and direct correctness, but its $\Theta(n^2)$ growth becomes a serious disadvantage as the dataset expands. Divide-and-Conquer improves the theoretical complexity to $\Theta(n \log n)$, making it much more suitable for large inputs, although sorting, recursion and auxiliary data structures increase implementation complexity.

The Hybrid method retains the scalable structure of Divide-and-Conquer while avoiding unnecessary recursive overhead on small partitions. Its main trade-off is the need to select a threshold, because an unsuitable threshold may reduce the expected performance benefit. Based on both theoretical reasoning and experimental measurements, the Hybrid method is justified as the preferred final solution when it consistently provides equal correctness with competitive or improved practical execution time.

12. Broader Considerations / SDG Relevance, wherever applicable

The project supports SDG 9: Industry, Innovation and Infrastructure by demonstrating how algorithmic efficiency can improve the analysis of infrastructure-scale spatial information. Faster processing of location data can support evidence-based planning, help identify potentially redundant facilities and improve the management of geographically distributed services.

The broader lesson of the project is that efficient algorithms are an important part of digital infrastructure. As public and industrial datasets continue to grow, scalable computational techniques become increasingly necessary for converting raw geographical information into useful planning knowledge.

13. Conclusion, Limitations and Possible Improvements

The study demonstrates that the choice of algorithm strongly affects the feasibility of large-scale closest-pair detection. Brute Force is useful as a correctness baseline but is limited by quadratic growth. Divide-and-Conquer provides a substantial theoretical improvement, and the Hybrid approach aims to improve practical performance by matching the solution method to the size of each subproblem.

The current study is limited to two-dimensional coordinates and Euclidean distance. Future work could include geographical distance models such as the Haversine formula, automatic threshold tuning, parallel execution, larger external datasets and integration with GIS-based visualisation systems

14. Individual Contribution of Group Members KS THARUNESHWAR

I have contributed to the **problem formulation, overall system design, algorithm development and analysis** of the project. I worked on understanding the **Real-Time Smart Parking Space Allocation problem**, implemented and analysed the **Greedy algorithm** for selecting suitable parking spaces, and developed the **Graph and Shortest Path algorithm** by modelling parking locations and routes. I also contributed to combining both algorithms, testing the system under different parking conditions, and analysing **execution time, search distance, parking-space utilization, memory usage and computational efficiency**. In addition, I worked on preparing the **problem statement, objectives, theoretical complexity analysis, system design, experimental results, graphs, conclusion, future improvements, documentation and presentation** of the project.

15. References, including datasets, software/tools and other sources used, wherever applicable

- References should include the Design and Analysis of Algorithms textbook prescribed for the course, official
- Python documentation, the source of the geospatial dataset used in the experiments, documentation for any libraries used and the official
- United Nations Sustainable Development Goal 9 resource. Use the citation style required by your institution consistently throughout the report.

16. One-Page Individual Reflection

This project provided me with a valuable opportunity to apply theoretical concepts from the Design and Analysis of Algorithms course to a practical large-scale data problem. Through the Closest Pair

of Points problem, I understood that selecting an appropriate algorithm is extremely important when the size of the input data increases. Initially, the Brute-Force approach appeared simple and easy to implement, but analysing its quadratic complexity demonstrated why it becomes inefficient for large datasets.

Working on the Divide-and-Conquer algorithm helped me understand how recursively dividing a problem into smaller subproblems can significantly improve performance. I also learned how the Master Theorem can be used to analyse recurrence relations and determine the theoretical efficiency of recursive algorithms. The recurrence $T(n) = 2T(n/2) + O(n)$ demonstrated how the DivideandConquer approach achieves $\Theta(n \log n)$ complexity.

The most interesting part of the project was developing the Hybrid algorithm. This approach demonstrated that theoretical efficiency alone is not always sufficient and that practical factors such as recursion overhead and implementation cost must also be considered. Combining Brute Force for small subproblems with Divide-and-Conquer for larger inputs provided a better balance between theoretical performance and practical execution speed.

The experimental evaluation improved my understanding of algorithm performance measurement. Measuring execution time, distance computations and memory usage helped me connect theoretical complexity analysis with actual program behaviour. I learned that hardware configuration and dataset characteristics can also influence experimental results.

Overall, this project improved my understanding of algorithm design, complexity analysis, optimisation and experimental evaluation. It also demonstrated how efficient computational methods can contribute to real-world infrastructure planning and support SDG 9, Industry.

E. Assessment Rubric

Criteria	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
----------	-----------	-----------	------	--------------	-------------------

Problem Understanding & Algorithmic Formulation	10	Clearly identifies the problem, requirements, inputs, outputs, constraints and computational challenges and formulates them appropriately for solution development.	Problem and major requirements are correctly formulated with minor gaps.	Basic formulation is provided, but requirements or constraints are partially identified.	Problem is poorly understood or inadequately formulated.
Algorithmic Solution Design	15	Designs a clear, logical and feasible solution strategy with appropriate algorithmic techniques, pseudocode/flow chart and well justified design decisions.	Appropriate solution design with minor gaps in logic, representation or justification.	Basic solution design is presented but lacks completeness or clear justification.	Solution design is inappropriate, incomplete or unclear.
Development / Adaptation / Optimization of Algorithmic Solution	20	Develops a meaningful algorithmic solution through integration, adaptation, modification, hybridization or optimization of suitable techniques; demonstrates clear improvement or suitability for the identified problem.	Develops an appropriate solution with meaningful adaptation or improvement, with minor limitations.	Solution demonstrates limited adaptation or development beyond standard implementation.	Merely reproduces an existing algorithm with little or no meaningful development or adaptation.

Implementation & Modern Tool Usage	15	Developed solution is functional with minor logical or efficiency issues. correctly and efficiently implemented using appropriate programming/computational tools and handles relevant input conditions reliably.	Implementation is functional with minor logical or efficiency issues.	Core implementation works but contains limitations or incomplete functionality.	Implementation is incomplete, unreliable or nonfunctional.
Efficiency, Testing & Validation	15	Performs appropriate complexity evaluation and systematic testing using suitable data/input sizes; validates correctness, efficiency and scalability with clear evidence.	Complexity evaluation and testing are mostly appropriate with minor gaps.	Basic testing and efficiency evaluation are provided but lack sufficient validation.	Testing, complexity evaluation or validation is inadequate or substantially incorrect.
Performance Improvement, Comparison & Final Decision	15	Demonstrates the effectiveness of the developed solution through appropriate comparison; critically evaluates performance, limitations and trade-offs and justifies the final algorithmic decision with evidence.	Good performance comparison and justification with minor limitations.	Basic comparison is provided, but improvement or final justification lacks depth.	Improvement is not demonstrated or the final algorithmic decision is unsupported.

Reflection: Algorithmic Design Decisions, SDG Relevance & Learning Outcomes	10	Thoughtful justification of algorithmic design and development decisions; clear connection to	Appropriate justification of algorithmic decisions; SDG relevance, challenges, improvements, and	Reflection is present but generic; limited justification of algorithmic decisions,	No genuine reflection is provided, or the reflection lacks meaningful
		relevant SDG(s); specific reflection on computational challenges, performance improvements, trade-offs, and learning gained.	learning are discussed but lack depth.	improvements, SDG relevance, or learning outcomes.	connection to algorithm development, SDG relevance, challenges, or learning outcomes.

F. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem Understanding & Algorithmic Formulation			L4	10
Algorithmic Solution Design			L4/L6	15
Development / Adaptation / Optimization of Algorithmic Solution			L5/L6	20
Implementation & Modern Tool Usage			L3/L6	15
Efficiency, Testing & Validation			L4/L5	15
Performance Improvement, Comparison & Final Decision			L4/L5	15

Reflection: Algorithmic Design Decisions, SDG Relevance & Learning Outcomes			L4/L5	10
---	--	--	-------	----

G. Faculty Evaluation & Continuous

Improvement CO Attainment

Performance Level	Criterion
Level 3	$\geq 70\%$
Level 2	60–69%
Level 1	50–59%
Level 0	$< 50\%$

Target CO Attainment: _____

Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well:

Strong conceptual understanding of foundational theoretical principles and core definitions.

Areas requiring improvement:

Difficulty applying theoretical concepts to complex, real-world engineering scenarios or novel problem statements.

Corrective / Improvement Action:

Provide supplementary practice sets containing application-based and analytical design problems.

Action to be implemented in the next offering:

- Integrate interactive case studies and mini-design projects directly into the active learning curriculum.
- Restructure continuous assessments to include higher-order thinking questions aligned with Bloom's Taxonomy.

